



Операционные системы

*«То, что мы делаем, уже сделано.
То, что мы собираемся сделать – сделано.
То, что мы представляем, что можно сделать, - сделано.
И даже то, что не представляем, - сделано тоже».*
Владимир Олегович Молодцов, доцент каф. ЛЭТИ

ОС как феномен программного обеспечения

ОС как система управления ресурсами: процессор, память, устройства, данные

Алгоритмы управления в ОС, сущность, аналоги

Архитектурные аспекты ОС, взаимодействие с «железом»

ОС как программная компонента: архитектура, структуры данных, алгоритмы, средства разработки. Внутреннее программирование в ОС.

ОС как сервис программирования и поддержки процессов



Операционные системы

Предмет. Определение ОС

Историческая и технологическая ретроспектива

ОС как система управления ресурсами

Ресурсы ВС: процессор, память, ВУ, данные.
Процессы и задачи как потребители ресурсов.
Оперативное и стратегическое управление ресурсами. Критерии эффективности ОС.

ОС как программная компонента
архитектура, структуры данных,
алгоритмы, средства разработки.
Внутреннее программирование в ОС





Операционные системы

Определение — ключевые свойства

Слой ПО, непосредственно работающий с архитектурой (на «голой» машине, standalone)

Без ОС — невозможность исполнения программ или деградация предоставляемых сервисов

Среда исполнения программ (**процессов, задач**)

Управление ресурсами ОС, распределение между процессами (**тактика, стратегия**). Вид ОС по режиму управления — **пакетная обработка, разделение времени, реальное время**

Компоненты ОС

- Собственно ОС — управление ресурсами и среда исполнения процессов
- Системные процессы (сеть, GUI, службы)
- Прикладные процессы и компоненты (интерфейс пользователя, среда исполнения программ)



Операционные системы

Историческая и технологическая ретроспектива

70-е годы. Разделение времени, независимая среда исполнения программы (приложения) - виртуальное АП. Текстовый терминал. Режим командной строки. Интерактивное программирование (**Basic**)

70-е годы. Эволюция ОС мини-ЭВМ от простых однопрограммных до полномасштабных ОС разделения времени

70-е годы. Эволюция mainframe в ВЦ коллективного пользования

70-е годы. Программирование как ремесло и технология. Структурное программирование. Язык «правильного программирования» - **Паскаль**. Машинно-независимый ассемблер — **классический Си. UNIX** — прецедент разработки ОС не на ассемблере

80-е годы. Технологическая революция Intel: функционально полный процессор в одном кристалле.



Операционные системы

Структура ОС как системы распределения ресурсов

Процесс — потребитель ресурсов системы

ОС - распределение ресурсов между процессами (тактическое управление и стратегическое планирование)

Основные типы ресурсов:

Процессор — диспетчеризация (переключение процессора между потоками (процессами), обработка прерываний, планирование процессов, синхронизация

Память — распределение виртуального и физического АП, загрузка/вытеснение/замещение процессов и страниц. Стратегии замещения

Устройства — согласование программных и аппаратных интерфейсов (драйверы), очереди в/в, обработка прерывания и прямой доступ к памяти (DMA)

Данные — файловая система: планирование носителя, размещение файлов, защита, восстановление

Пользователь — поток данных от устройств



Операционные системы

ОС как система массового обслуживания

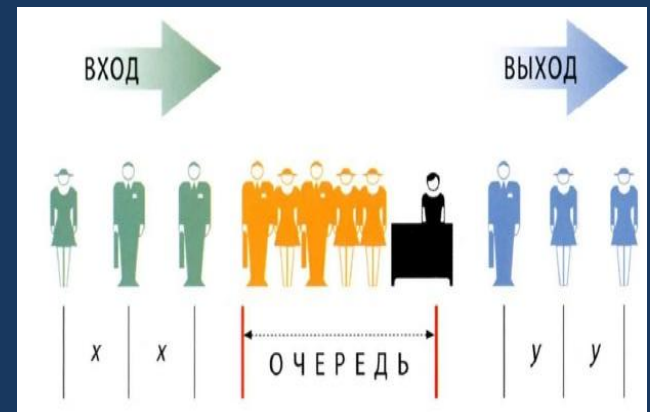
Теория массового обслуживания

Предпосылка: развитие телефонии (АТС) — определение количества обслуживающих приборов (коммутаторов), размеров очередей при случайном потоке заявок (звонков) и случайном времени их обслуживания. Расчет очередей и задержек

Базируется на теории вероятности

Применима к любой системе с потоками запросов к системам обслуживания или ресурсам

Получила развитие в 70-80 г. - оценка производительности ВС





Операционные системы

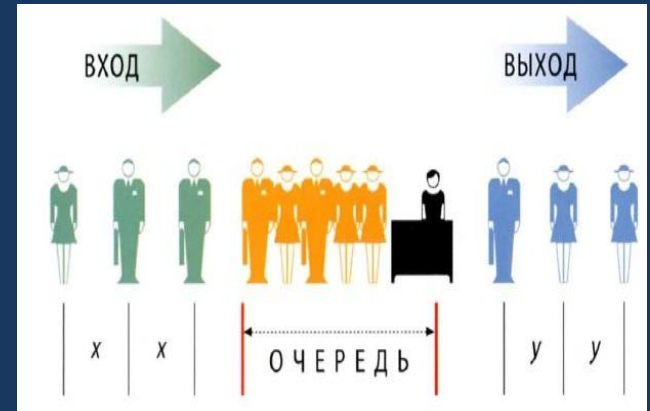
ОС как система массового обслуживания

Критерии качества системы:

- % использования ресурсов
- Время ожидания запросов (в т.ч. субъективное восприятие)

Виды ОС по режиму управления

- Ресурсами:
 - ОС **пакетной обработки** — макс. загрузка ресурсов
 - ОС **разделения времени** — «справедливое» (равномерное) распределение ресурсов
 - ОС **реального времени** - обеспечение заданной реакции на внешние события





Операционные системы

ОС пакетной обработки

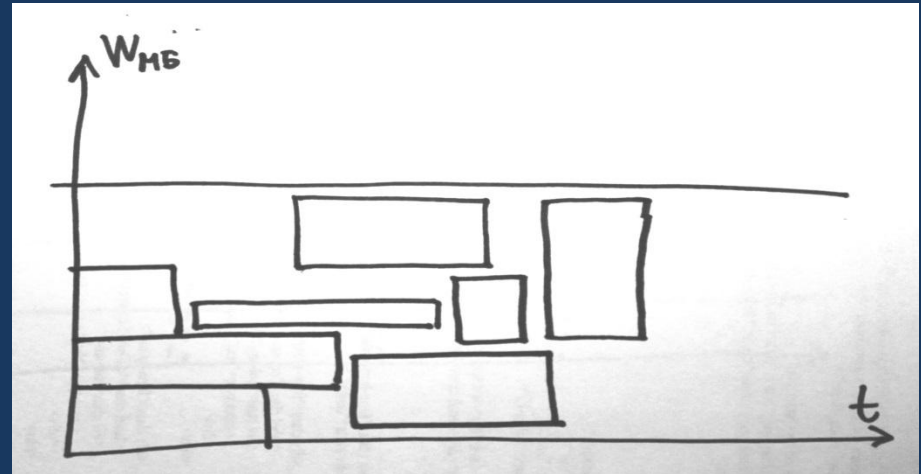
Критерии качества системы: макс. % использования ресурсов (процессор, память, ВУ)

- ОС 60-х годов для mainframe
- Планирование заданий в мультипроцессорных системах

Необходимые компоненты ОС:

- Разделение времени процессора
- Планирование размещения в памяти
- Очереди в/в

Планирование времени запуска процессов (расписание)





Операционные системы

ОС разделения времени

Критерии качества системы: равномерное (справедливое) распределение ресурсов между процессами

- Практически все интерактивные ОС являются ОС разделения времени

Необходимые компоненты ОС:

- Квантование, круговая диспетчеризация
- Очереди процессов (потоков)



«Справедливость»: приоритетная круговая диспетчеризация (фоновые задания, интерактивные, реального времени)



Операционные системы

ОС реального времени

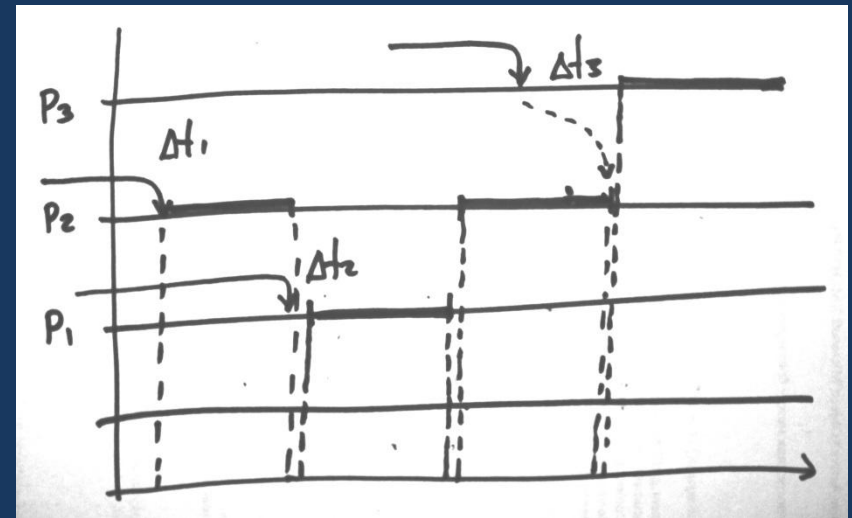
Критерии качества системы: обеспечение гарантированного времени реакции на внешние события (прерывания)

Необходимые компоненты ОС:

Вытесняющая (приоритетная) многозадачность

Обработка прерываний

Слежение за временем (таймеры)



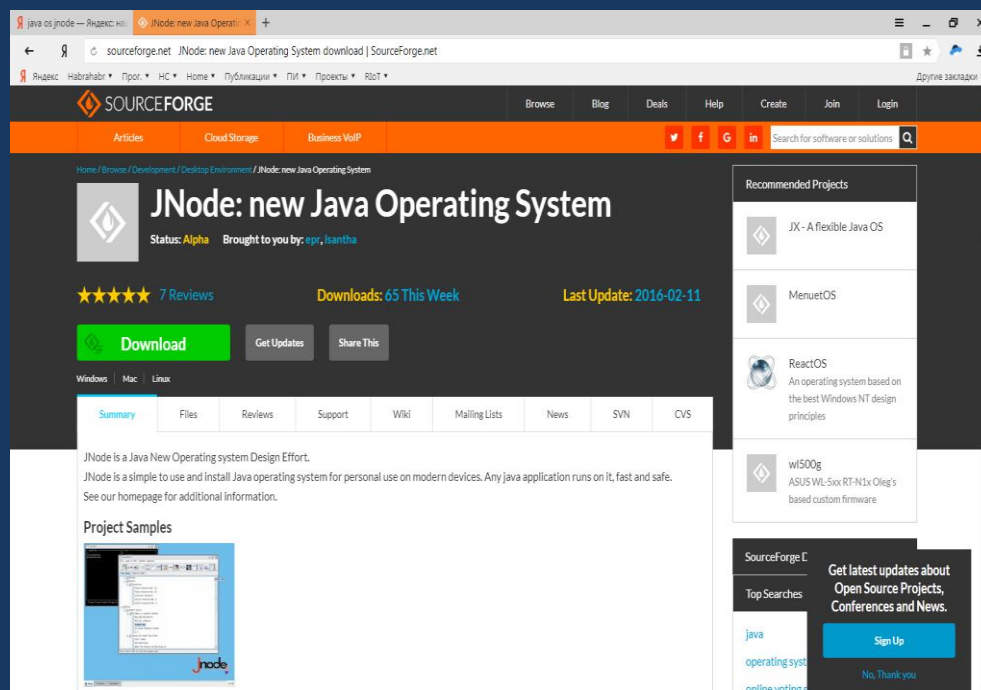
Система обработки прерываний в ОС является компонентой реального времени с вытесняющей диспетчеризацией



Операционные системы

Многообразие подходов к реализации ОС

- «корпоративная» ОС Windows
- Семейства ОС на основе открытого кода: (UNIX, LINUX, FreeBSD)
- Специализированные ОС на основе ядра LINUX
- ОС реального времени для микроконтроллеров (FreeRTOS)
- Оригинальные проекты: ОС для среды исполнения байт-кода Java





Операционные системы

PDP-8. ОС на левой коленке



- Разделение времени
 - «Защищенный режим»
 - Псевдоустройства
 - Файловая система и файловый сервер
 - До 8 пользователей
 - Режимы 8-Basic(Focal) и разработка
- и все это в 32К 12-разрядных слов**





Операционные системы

FreeRTOS — операционная система для микроконтроллеров

Платформенно-независимые компоненты

- include
- portable
- croutine.c
- event_groups.c
- list.c
- queue.c
- readme.txt
- tasks.c
- timers.c

- croutine.h
- deprecated_definitions.h
- event_groups.h
- FreeRTOS.h
- list.h
- mpu_prototypes.h
- mpu_wrappers.h
- portable.h
- projdefs.h
- queue.h
- semphr.h
- StackMacros.h
- stdint.readme
- task.h
- timers.h

```
UBaseType_t uxListRemove( ListItem_t * const pxItemToRemove )
{
    /* The list item knows which list it is in. Obtain the list from the
    item. */
    List_t * const pxList = ( List_t * ) pxItemToRemove->pvContainer;

    pxItemToRemove->pxNext->pxPrevious = pxItemToRemove->pxPrevious;
    pxItemToRemove->pxPrevious->pxNext = pxItemToRemove->pxNext;

    /* Only used during decision coverage testing. */
    mtCOVERAGE_TEST_DELAY();
}
```



Операционные системы

FreeRTOS — операционная система для микроконтроллеров

Платформенно-зависимые компоненты

```
macro portSAVE_CONTEXT

    /* Save the LR and SPSR onto the system mode stack before
    system mode to save the remaining system mode registers. */
    SRSDB    sp!, #SYS_MODE
    CPS      #SYS_MODE
    PUSH     {R0-R12, R14}

    /* Push the critical nesting count. */
    LDR      R2, ulCriticalNestingConst
    LDR      R1, [R2]
    PUSH     {R1}

    /* Does the task have a floating point context?
    ulPortTaskHasFPUContext is 0 then no. */
    LDR      R2, ulPortTaskHasFPUContextConst
    LDR      R3, [R2]
    CMP      R3, #0
```

port.c
portASM.S
portmacro.h

```
/* Next the return address, which in this case is the start
*pxTopOfStack = ( StackType_t ) pxCode;
pxTopOfStack--;

/* Next all the registers other than the stack pointer. */
*pxTopOfStack = ( StackType_t ) portTASK_RETURN_ADDRESS;
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) 0x12121212; /* R12 */
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) 0x11111111; /* R11 */
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) 0x10101010; /* R10 */
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) 0x09090909; /* R9 */
pxTopOfStack--;
*pxTopOfStack = ( StackType_t ) 0x08080808; /* R8 */
pxTopOfStack--;
```